GHOSTWIRE

APK Compilation Guide · v1.1.0
Reproducible builds on Google Colab, Ubuntu and Docker

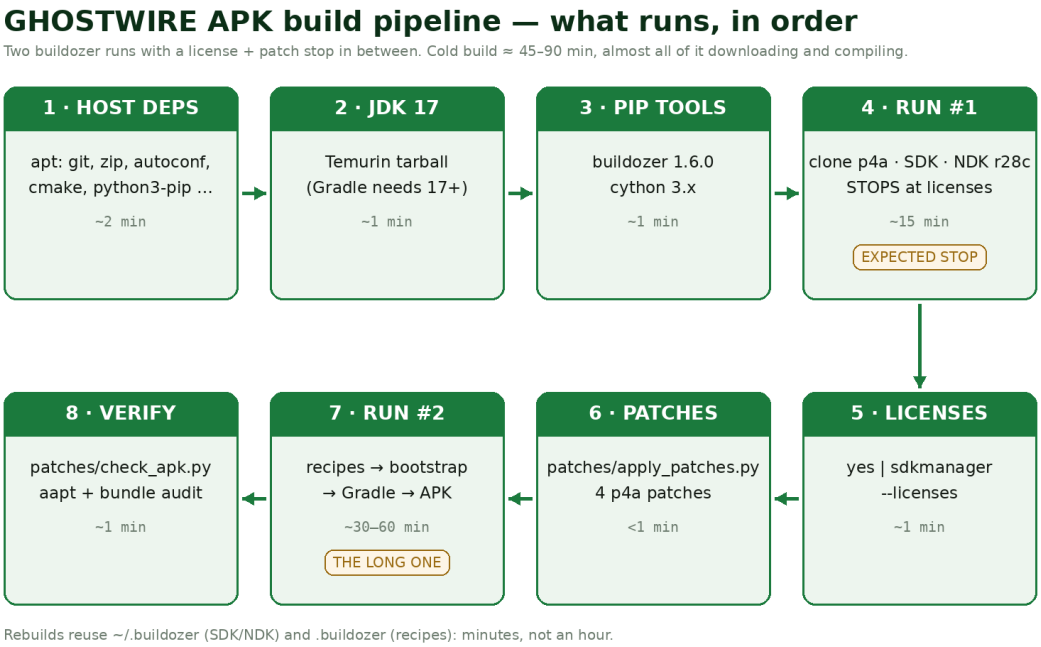


Figure 1 — The whole process on one page. Details inside.

Every command in this book ran verbatim to ship the release APK.
Nothing is paraphrased from memory.

developed by @Ariko & Arena

Contents

1 What you are building 3

2 How the build works 4

3 Project checklist 4

4 Path A — Google Colab 5

5 Path B — Ubuntu 22.04 / 24.04 (PC or VM) 7

6 Path C — Docker (any OS that runs Docker) 8

7 The 4 p4a patches — what, why, and proof 9

8 Verification — prove the APK before you ship it 10

9 Error catalog — every failure I hit, decoded 11

10 Rebuilds and iteration 13

11 Pre-flight checklist and FAQ 14

1 What you are building

Input: the ghostwire/ project folder (Python source + buildozer.spec). Output: bin/ghostwire-1.1.0-arm64-v8a-debug.apk (~31 MB), installable on any Android 10+ arm64 phone. The tool that turns one into the other is buildozer, which drives python-for-android (p4a): p4a cross-compiles Python + every native library for ARM64, wraps them in an SDL2 Android bootstrap, Gradle assembles the APK.

1.1 Cost of a cold build

Resource	Budget	Notes
Time	45–90 min	Downloads ~15 min, recipe compiles ~30–60 min on 2 vCPUs, Gradle ~3 min. Rebuilds take minutes.
Disk	15 GB free	~3 GB SDK/NDK + ~8 GB recipes/dist. Docker hosts: check before you start (E10).
RAM	4 GB ideal	2 GB works with a 3 GB swapfile (§5.5). Colab's 12 GB needs nothing.

1.2 Version manifest — the exact stack that shipped

These are not recommendations. They are the versions the release APK was compiled with; replicate them first, experiment later.

Component	Version	How it arrives
Host OS	Debian 13 x86_64 (sandbox); Ubuntu 22.04/24.04 equivalent	your machine / Colab / image
Host Python	3.13	system python3
buildozer	1.6.0	pip
Cython	3.x (pip latest)	pip; fallback pin in §9 E12
JDK	Temurin 17.0.20.1+1	tarball (§4–6) or apt (§5.2)
python-for-android	master @ 58d21141	buildozer clones it
Android NDK	r28c	buildozer downloads it
SDK platform / build-tools	android-34 / 34.x	sdkmanager
platform-tools	r37.0.1	sdkmanager
Apache ANT	1.9.4	buildozer downloads it
Gradle	8.14.3	gradlew downloads it
Target Python (on Android)	CPython 3.14.2	p4a recipe
App requirements	kivy==2.3.1, kivymd==1.2.0, pycryptodome	buildozer.spec

Component	Version	How it arrives
APK floor	minSdk 24 / targetSdk 34 / arm64-v8a	buildozer.spec

2 How the build works

2.1 The two-run shape

You run buildozer twice. Run #1 clones p4a and downloads the SDK/NDK, then halts at the license prompt (E2) — that stop is expected and useful: it gives you a complete p4a tree to patch and an SDK tree to license. Then you accept licenses, apply the 4 GHOSTWIRE patches, and run #2 compiles everything and assembles the APK.

2.2 Directory map — know where things land

Path	Owner	Contents
~/.buildozer/android/platform/	buildozer	android-sdk, android-ndk-r28c, apache-ant. Reused by every build.
ghostwire/.buildozer/	buildozer+p4a	p4a clone, compiled recipes, dist skeleton, staged app. Wiped by `buildozer android clean`.
ghostwire/bin/	buildozer	The finished APK(s).
~/jdk-17*/	you	Temurin JDK (tarball path).

Golden rule — Patches live in ghostwire/.buildozer, which `clean` deletes and fresh clones lack. Any time that tree is (re)created, re-run patches/apply_patches.py before the compiling run. The script is idempotent — when in doubt, just run it again.

2.3 The environment block — used on all three paths

Every shell that runs buildozer needs this (Colab sets it once via os.environ in §4 step 5):

```
export JAVA_HOME=$(ls -d $HOME/jdk-17* | head -1)
export PATH="$JAVA_HOME/bin:$HOME/.local/bin:$PATH"
export GRADLE_OPTS="-Xmx1500m -XX:MaxMetaspaceSize=512m"
export KIVY_NO_ARGS=1
javac -version # must print javac 17...
keytool -help 2>&1 | head -1 # must exist (buildozer needs it)
```

Why each line: JAVA_HOME+PATH force Java 17 everywhere (E4); $HOME/.local/bin keeps p4a's pip-installed helpers on PATH; GRADLE_OPTS caps Gradle's heap so small machines survive; KIVY_NO_ARGS stops Kivy parsing CLI flags during the build.

3 Project checklist

The build needs this layout (names matter, the rest of the repo rides along harmlessly):

```
ghostwire/
  main.py icon.png presplash.png presplash.json
  buildozer.spec # §1.2 floor + requirements
```

```
gw/__init__.py gw/crypto.py gw/net.py gw/ui.py
gw/_curve25519.py
patches/apply_patches.py # the 4 p4a patches
patches/check_apk.py # release verification
docker/Dockerfile docker/build.sh # path C only
colab/GHOSTWIRE_APK_Build.ipynb # path A only
```

Key buildozer.spec lines and what they do:

Line	Meaning
requirements = python3,kivy==2.3.1,kivymd==1.2.0,pycryptodome	Pinned recipes p4a must compile. Exact pins = reproducible.
android.api = 34 / android.minapi = 24	targetSdk 34 (no old-SDK warning), minSdk 24 (Android 7+ install floor; supported from 10+).
android.archs = arm64-v8a	Modern phones only; keeps the APK to one ABI (~31 MB).
source.exclude_dirs / exclude_patterns	Keeps docs/, shots/, tests and relay_server.py OUT of the APK (verified §8).
android.presplash_lottie = ./presplash.json	Animated splash; patch 2+4 guarantee a static frame fallback.
warn_on_root = 0	Docker builds run as root; silences the warning without changing behaviour.

4 Path A — Google Colab

Best when your own machine is slow, full, or Windows/macOS without Docker. Free CPU runtime is enough (12 GB RAM, no swap needed). Upload colab/GHOSTWIRE_APK_Build.ipynb (File → Upload notebook) and Runtime → Run all — or follow the cells below, which are its exact contents. Keep the tab open; the 12-hour limit is far beyond the ~1 hour a cold build needs.

4.1 Cell 1 — system packages (~2 min)

```
!apt-get update -qq && apt-get install -qq -y git zip unzip autoconf libtool pkg-config zlib1g-dev libncurses-dev cmake libffi-dev libssl-dev python3-pip
```

4.2 Cell 2 — JDK 17 Temurin (~1 min)

Colab ships Java 11; Gradle 8 + AGP demand 17 (E4). This is the same tarball the release used:

```
!cd /root && curl -sL -o jdk17.tar.gz "https://api.adoptium.net/v3/binary/latest/17/ga/linux/x64/jdk/hotspot/normal/eclipse"
!tar xzf jdk17.tar.gz && /root/jdk-17*/bin/java -version
```

```
openjdk version "17.0.20.1" ... (any 17.x is fine)
```

4.3 Cell 3 — buildozer + cython

```
!pip3 install -q buildozer cython && buildozer --version && cython --version
```

```
Buildozer 1.6.0
```

4.4 Cell 4 — get the source

```
!cd /content && ( [ -d ghostwire/.git ] && git -C ghostwire pull || git clone
https://github.com/YOU/ghostwire.git ) && ls ghostwire
```

Replace the URL with your repo. Zip alternative: upload ghostwire.zip to /content, then `!cd /content && rm -rf ghostwire && unzip -q ghostwire.zip`.

4.5 Cell 5 — environment (Python cell, NOT shell)

Each `!` shell forgets exports; os.environ persists for the whole session — that is why this cell is Python:

```
import os
os.environ["JAVA_HOME"] = "/root/jdk-17.0.20.1+1"
os.environ["PATH"] = "/root/jdk-17.0.20.1+1/bin:" + os.environ["PATH"]
os.environ["GRADLE_OPTS"] = "-Xmx1500m -XX:MaxMetaspaceSize=512m"
os.environ["KIVY_NO_ARGS"] = "1"
```

> Version drift — If Adoptium has released a newer 17.x, the folder name differs. List /root/jdk-17* and adjust the two paths — everything else is identical.

4.6 Cell 6 — run #1: fetch platform (stops at licenses)

```
!cd /content/ghostwire && buildozer android debug > /tmp/b1.log 2>&1 || true
!tail -4 /tmp/b1.log
!grep -q "Aidl not found" /tmp/b1.log && echo "OK: reached license stage (expected stop)"
```

The `|| true` is deliberate: run #1 is *supposed* to fail here (E2). The grep proves it failed at the right place. Downloads take ~15 min on Colab's network.

4.7 Cell 7 — accept SDK licenses (~1 min)

```
!SM=/root/.buildozer/android/platform/android-sdk/tools/bin/sdkmanager; yes | $SM
--sdk_root=/root/.buildozer/android/platform/android-sdk --licenses 2>&1 | tail -2

Accept? (y/N): All SDK package licenses accepted
```

The `yes |` answers every prompt; `--sdk_root` is mandatory in this SDK layout (E3). Eight licenses accept; the build-tools that run #1 missed will now install on run #2.

4.8 Cell 8 — apply the 4 p4a patches

```
!cd /content/ghostwire && python3 patches/apply_patches.py
```

Expect five OK/SKIP lines (§7). Run #1 created the p4a tree; this is the only window where patching is needed or possible.

4.9 Cell 9 — run #2: the build (30–60 min)

```
!cd /content/ghostwire && buildozer android debug 2>&1 | tee /tmp/b2.log | grep --line-buffered -E
"Downloading|Recipe |BUILD|available in the bin" | tail -60
!ls -la /content/ghostwire/bin/
```

The grep streams *milestones* live (otherwise the cell looks dead for an hour) while tee keeps the full log at /tmp/b2.log for debugging (§9). Success ends with:

```
# APK ghostwire-1.1.0-arm64-v8a-debug.apk available in the bin directory
-rw-r--r-- 1 root root 30730061 ... ghostwire-1.1.0-arm64-v8a-debug.apk
```

4.10 Cell 10 — verify + download

```
!cd /content/ghostwire && python3 patches/check_apk.py bin/ghostwire-1.1.0-arm64-v8a-debug.apk
from google.colab import files
files.download("/content/ghostwire/bin/ghostwire-1.1.0-arm64-v8a-debug.apk")
```

check_apk prints ~20 PASS lines (§8), then the APK downloads to your computer. Sideload and go.

> Colab amnesia — A recycled runtime loses EVERYTHING (SDK, recipes, patches). Recovery is just Runtime → Run all again (~1 h). Optionally mount Drive and copy bin/*.apk there at the end so the artifact survives even if the download is missed.

5 Path B — Ubuntu 22.04 / 24.04 (PC or VM)

The native path. Steps mirror Colab, with sudo and a couple of Ubuntu conveniences. Assumes an x86_64 machine with 15 GB free disk.

5.1 System packages

```
sudo apt-get update && sudo apt-get install -y git zip unzip autoconf libtool pkg-config
zlib1g-dev libncurses-dev cmake libffi-dev libssl-dev python3-pip
```

5.2 JDK 17 — two good options

Option 1 — distro package (simplest on Ubuntu; it exists here, unlike some Debian spins):

```
sudo apt-get install -y openjdk-17-jdk-headless
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
$JAVA_HOME/bin/javac -version # expect javac 17...
```

Option 2 — Temurin tarball (byte-identical to the release environment):

```
cd ~ && curl -sL -o jdk17.tar.gz
"https://api.adoptium.net/v3/binary/latest/17/ga/linux/x64/jdk/hotspot/normal/eclipse"
tar xzf jdk17.tar.gz && ~/jdk-17*/bin/java -version
```

5.3 Python build tools

```
pip3 install --break-system-packages -q buildozer cython
buildozer --version # expect 1.6.0
```

The --break-system-packages flag is required on Ubuntu 24.04+ (E11); on 22.04 it is accepted harmlessly. Prefer a venv? Create one and drop the flag — but then activate it in EVERY shell that runs buildozer.

5.4 Source + environment + run #1 + licenses + patches

```
cd ~ && git clone https://github.com/YOU/ghostwire.git && cd ghostwire
export JAVA_HOME=$(ls -d $HOME/jdk-17* /usr/lib/jvm/java-17-openjdk-amd64 2>/dev/null | head -1)
export PATH="$JAVA_HOME/bin:$HOME/.local/bin:$PATH"
export GRADLE_OPTS="-Xmx1500m -XX:MaxMetaspaceSize=512m"
export KIVY_NO_ARGS=1
buildozer android debug 2>&1 | tee /tmp/buildozer.log # stops at licenses (E2)
SM=$HOME/.buildozer/android/platform/android-sdk/tools/bin/sdkmanager
yes | $SM --sdk_root=$HOME/.buildozer/android/platform/android-sdk --licenses
python3 patches/apply_patches.py
```

Consider appending the four exports to ~/.bashrc so every terminal is build-ready. Then run #2 and verify exactly as in §4.9–4.10 (without the `!` and files.download).

5.5 Low-RAM machines (2 GB): add swap first

The kivy recipe's -O3 clang lines will OOM-thrash a 2 GB box (E6). This exact sequence saved the release build:

```
sudo fallocate -l 3G /swapfile && sudo chmod 600 /swapfile && sudo mkswap /swapfile && sudo swapon /swapfile
free -m | head -3 # Swap: ~3071
```

Keep the swapfile until the build finishes; remove afterwards with `sudo swapoff /swapfile && sudo rm /swapfile` if desired.

6 Path C — Docker (any OS that runs Docker)

Fully scripted: the image carries the toolchain, build.sh performs all five stages, and the APK lands back on your host. Files live in the repo under docker/ and are printed here verbatim.

6.1 Prerequisites

- Docker with ≥15 GB free image/container space.
- Linux/macOS/Windows host — the container is always Ubuntu 24.04 x86_64. Apple Silicon: add --platform linux/amd64 to every docker command (SDK/NDK are x86_64-only; emulation is slower but works).

6.2 docker/Dockerfile (as shipped)

```
FROM ubuntu:24.04
ENV DEBIAN_FRONTEND=noninteractive \
    JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64 \
    GRADLE_OPTS="-Xmx1500m -XX:MaxMetaspaceSize=512m" \
    KIVY_NO_ARGS=1
ENV PATH="$JAVA_HOME/bin:/root/.local/bin:$PATH"
RUN apt-get update && apt-get install -y --no-install-recommends \
        git zip unzip autoconf libtool pkg-config \
        zlib1g-dev libncurses-dev cmake libffi-dev libssl-dev \
        python3-pip openjdk-17-jdk-headless \
    && rm -rf /var/lib/apt/lists/* \
    && pip3 install --break-system-packages --no-cache-dir \
        buildozer cython \
    && buildozer --version && javac -version
WORKDIR /src
COPY . /src/
CMD ["/src/docker/build.sh"]
```

Notes: Ubuntu 24.04 supplies OpenJDK 17 via apt, so no tarball is needed inside the image; .dockerignore keeps SDK-sized junk and the 300-page PDF out of the build context.

6.3 docker/build.sh — the five stages (as shipped)

```
#!/bin/bash
# [1/5] run #1 (license stop expected)
buildozer android debug > /tmp/buildozer1.log 2>&1 || true
grep -qa "Aidl not found" /tmp/buildozer1.log || exit 1
# [2/5] licenses
```

```
SM=/root/.buildozer/.../android-sdk/tools/bin/sdkmanager
yes | "$SM" --sdk_root=/root/.buildozer/.../android-sdk --licenses
# [3/5] patches
python3 patches/apply_patches.py /src || exit 1
# [4/5] run #2 (recipes + gradle)
buildozer android debug 2>&1 | tee /tmp/buildozer2.log | tail -3
# [5/5] verify, fail loudly otherwise
python3 patches/check_apk.py "${APK[0]}" || exit 1
```

Each stage gates the next: a wrong run-#1 failure, a patch miss, or a failed audit aborts with the log path — never a silent bad APK.

6.4 Build, run, collect

```
cd ghostwire && docker build -t ghostwire-builder -f docker/Dockerfile .
docker run --rm -v "$PWD/bin:/src/bin" ghostwire-builder
ls -la bin/ # ghostwire-1.1.0-arm64-v8a-debug.apk (~31 MB)
```

The bind mount is what carries the APK out — without it the artifact dies with the container. First run ≈1 h; for minute-scale rebuilds, persist the SDK + recipes across runs:

```
docker volume create gw-android-sdk
docker run --rm -v "$PWD/bin:/src/bin" -v gw-android-sdk:/root/.buildozer -v
"$PWD/.buildozer:/src/.buildozer" ghostwire-builder
```

> Root is fine here — The container runs as root and buildozer.spec sets warn_on_root = 0, so no prompt or warning interrupts the scripted flow.

7 The 4 p4a patches — what, why, and proof

python-for-android ships a generic bootstrap; GHOSTWIRE needs four surgical changes. patches/apply_patches.py applies them to the fresh p4a clone (verified against master @ 58d21141) and skips any already present.

#	File	Change	If skipped
1	_sdl_common/.../AndroidManifest.tmpl.xml	android:windowSoftInputMode="adjustPan" on the entrypoint activity.	Soft keyboard covers the chat composer.
2	common/build/build.py	Copy the static presplash frame even in the lottie branch.	No fallback image (§8 asset check fails).
3	common/build/templates/kivy-presplash.jpg	Replaced with the project's presplash.png art.	Generic Kivy splash art.
4	sdl2/.../PythonActivity.java	attachLottieFailureListener + swapToStaticSplash (+ imports): a broken animation swaps to the static frame via reflection.	Black seconds when lottie fails; risk of the E5 arity bug if hand-edited.

7.1 Running it + expected output

```
cd ~/ghostwire && python3 patches/apply_patches.py

--- 1/4 manifest adjustPan
```

```
    OK
--- 2/4 static presplash in lottie branch
    OK
--- 3/4 presplash art
    OK (copied .../presplash.png)
--- 4a/4 Java imports
    OK
--- 4b/4 Java failure-swap
    OK
ALL PATCHES APPLIED (or already present).
```

Re-running prints SKIP lines — that is the idempotency check, not an error. ANCHOR NOT FOUND means your p4a is newer than 58d21141 and moved the surrounding code: inspect the named file by hand rather than forcing it.

8 Verification — prove the APK before you ship it

Run the auditor (it finds aapt inside the SDK buildozer installed):

```
cd ~/ghostwire && python3 patches/check_apk.py bin/ghostwire-1.1.0-arm64-v8a-debug.apk
```

```
aapt: .../android-sdk/build-tools/34.0.0/aapt
apk : bin/ghostwire-1.1.0-arm64-v8a-debug.apk (29 MB)
PASS versionName is 1.1.0
PASS minSdk 24 (0x18)
PASS targetSdk 34 (0x22)
PASS allowBackup=false
PASS adjustPan (0x20)
PASS static presplash bundled
PASS lottie splash bundled
PASS ships on_inbound_request
PASS ships REQUESTS A CHAT
PASS ships relay_connect
PASS ships WAN RELAY
PASS ships ttl_label
PASS ships REQUEST_TIMEOUT
PASS ships 1.1.0
PASS no residue: SessionChip ... (x7)
PASS docs/ excluded
PASS tests excluded
PASS relay_server.py excluded (stays server-side)
RESULT: OK
```

8.1 The same checks by hand (when you want to see it)

```
AAPT=$(ls $HOME/.buildozer/android/platform/android-sdk/build-tools/*/aapt | head -1)
$AAPT dump xmltree bin/*.apk AndroidManifest.xml | grep -i
"softInputMode\|allowBackup\|targetSdkVersion\|minSdkVersion\|versionName"
unzip -l bin/*.apk | grep -E "presplash|splashscreen"
```

```
A: android:versionName(0x0101021c)="1.1.0"
A: android:minSdkVersion(0x0101020c)=(type 0x10)0x18
A: android:targetSdkVersion(0x01010270)=(type 0x10)0x22
A: android:allowBackup(0x01010280)=(type 0x12)0x0
A: android:windowSoftInputMode(0x0101022b)=(type 0x11)0x20
    17519 ... res/drawable/presplash.jpg
```

```
6916 ... res/raw/splashscreen.json
```

Read it as: right version, Android 7+ installs / modern target (no stale-SDK warning), no ADB-backup key exfil, keyboard pans, both splash assets present. The bytecode half (markers in/out) is what check_apk.py's private.tar audit adds on top.

9 Error catalog — every failure I hit, decoded

Rule zero: buildozer prints `gradlew failed!` for everything. The real cause is always a few dozen lines up:

```
grep -a "What went wrong" -A 12 /tmp/buildozer.log | tail -20
```

ID	Signature line	One-line fix
E1	Java compiler (javac) not found	JDK bin dir on PATH.
E2	Aidl not found, please install it.	Accept SDK licenses.
E3	sdkmanager: Could not create settings	Add --sdk_root.
E4	AGP requires Java 17 ... using Java 11	Build under JDK 17.
E5	getIdentifier ... cannot be applied	Use the patch script.
E6	internal compiler error: Killed / thrash	Add swap / RAM.
E7	gradlew failed! (generic)	Grep “What went wrong”.
E8	Permission denied / missing 'server' JVM	Re-extract JDK.
E9	Colab tab died mid-build	Rerun all; keep tab open.
E10	Docker: no space / exec format error	Free 15 GB / amd64.
E11	externally-managed-environment (pip)	Add the pip flag.
E12	Cythonize errors under kivy	Pin cython==0.29.37.

9.1 E1 — javac / keytool not found

```
# Search for Java compiler (javac)
# Java compiler (javac) not found, please install it.
```

Meaning: buildozer scans PATH and found no JDK. Even with Java installed, the check fails if its bin/ is not on PATH — this bit me on every fresh shell. Fix: export the §2.3 block (mind the order: JDK first) and re-run. Verify with `javac -version` + `keytool -help` before rebuilding.

9.2 E2 — Aidl not found (the license stop)

```
# Check that aidl can be executed
# build-tools folder not found .../android-sdk/build-tools
# Aidl not found, please install it.
```

Meaning: build-tools never installed because the SDK licenses were never accepted — buildozer cannot click through the interactive prompt, so installation silently skipped. It is *not* fixed by apt. Fix (§4.7/§5.4): `yes | sdkmanager --sdk_root=... --licenses`, confirm “All SDK package licenses accepted”, re-run buildozer.

9.3 E3 — sdkmanager settings crash

```
Warning: Could not create settings
java.lang.IllegalArgumentException
    at ...SdkManagerCliSettings.<init>...
```

Meaning: this SDK layout (tools/ instead of cmdline-tools/latest/) forces you to pass --sdk_root explicitly. Fix: add the flag exactly as in §4.7. Any sdkmanager invocation in this guide without it will die the same way.

9.4 E4 — Gradle wants Java 17, found Java 11

```
* What went wrong:
Android Gradle plugin requires Java 17 ... currently using Java 11
```

Meaning: Gradle/AGP 8 run only on 17+. The system java (often 11 on Colab/older images) won the PATH race. Fix: the §2.3 block with JAVA_HOME + JDK-first PATH, exported in the SAME shell/cell that runs buildozer — and on Colab, via os.environ (§4.5), not `!export`.

9.5 E5 — getIdentifier arity (patch bug class)

```
PythonActivity.java:419: error: method getIdentifier in class
ResourceManager cannot be applied to given types;
    required: String,String
    found: String,String,String
```

Meaning: p4a's ResourceManager.getIdentifier takes 2 args; a 3-argument call (this guide's author's own first draft) fails compileDebugJavaWithJavac. Fix: use patches/apply_patches.py, which ships the corrected 2-arg call — never hand-edit the Java unless the script reports anchor drift (§7.1).

9.6 E6 — compiler Killed / box thrashing

```
c++: internal compiler error: Killed (program cc1plus)
# kswapd at 40%+, load 10+, everything frozen
```

Meaning: the OOM killer shot a clang during kivy's -O3 compile — 2 GB RAM with no swap cannot hold it. Fix: the §5.5 swapfile (release-proven on a 2 GB box), close browsers, or raise the Docker/VM memory to 4 GB+. The build resumes where it died: just re-run buildozer.

9.7 E7 — generic gradlew failure

`ERROR: .../dists/ghostwire/gradlew failed!` alone diagnoses nothing; twice it sent me hunting the wrong layer. Always pull the Gradle block first with the rule-zero grep at the top of this chapter — E4 and E5 both surfaced exactly that way.

9.8 E8 — JDK lost its edge (restores, copies, snapshots)

```
.../bin/java: Permission denied
Error: missing `server' JVM at `.../lib/server/libjvm.so'
```

Meaning: exec bits stripped (first line) or the tree partially copied (second — a snapshot once kept 95 of ~300 MB). Fix: `chmod -R +x` the bin/ dir for the first; for the second, delete and re-extract the tarball — never trust a half JDK.

9.9 E9 — Colab interruptions

A closed laptop, a throttled tab, or the 12-hour cap kills the VM and with it ~/.buildozer and the p4a tree — there is no resume. Prevention: keep the tab focused during run #2 (its milestone grep keeps output alive); recovery: Runtime → Run all from the top (~1 h). Save finished APKs to Drive, not just downloads (§4.10).

9.10 E10 — Docker space / architecture

```
no space left on device
exec /src/docker/build.sh: exec format error
```

Meaning 1: image + SDK + recipes exceed free Docker disk — free 15 GB (`docker system prune -a` if desperate) and rebuild. Meaning 2 (Apple Silicon): the image built for arm64 but the NDK is x86_64-only — rebuild AND run with `--platform linux/amd64`.

9.11 E11 — pip refuses (externally managed)

```
error: externally-managed-environment
This environment is externally managed
```

Meaning: Ubuntu 24.04+/Debian 13 PEP 668 guard. Fix: add `--break-system-packages` (container/Colab/throwaway VM — safe) or use a venv you activate in every build shell.

9.12 E12 — Cython version skew (fallback pin)

The release compiled clean on Cython 3.x (pip latest). If a future Cython breaks kivy 2.3.1's cythonize step, pin the legacy line kivy grew up with, then rebuild:

```
pip3 install --break-system-packages -q "cython==0.29.37"
cython --version # expect 0.29.37
```

10 Rebuilds and iteration

Run #2 is incremental: SDK/NDK (in ~/.buildozer) and compiled recipes (in ghostwire/.buildozer) are reused, so a code edit rebuilds in minutes:

```
cd ~/ghostwire # env block active (§2.3)
# edit gw/*.py, main.py, presplash.json ...
buildozer android debug 2>&1 | tee /tmp/rebuild.log | tail -3
python3 patches/check_apk.py bin/ghostwire-1.1.0-arm64-v8a-debug.apk
```

You changed	Do this	Cost
App Python / art / json	Re-run buildozer.	Minutes.
buildozer.spec requirements	Re-run; new recipes compile.	Minutes–tens of minutes.
version = line	Re-run; filename follows it.	Minutes.
Need a clean tree	`buildozer android clean`, then run #1 → licenses (kept) → patches → run #2.	Full cold build; patches MUST be re-applied.

The clean trap — `buildozer android clean` deletes the p4a tree — and your four patches with it. The very next compiling run without re-patching ships a keyboard-covering, splash-fragile APK that still installs fine. check_apk.py exists to catch exactly that.

11 Pre-flight checklist and FAQ

11.1 Pre-flight (2 minutes, saves hours)

- ☐ 15 GB free disk (E10) · 4 GB RAM or swapfile ready (E6).
- ☐ Env block exported in THIS shell (E1/E4): javac -version prints 17, keytool exists.
- ☐ On a fresh p4a tree: licenses accepted (E2) AND patches applied (§7) before the compiling run.
- ☐ Colab: tab stays open, files.download cell kept (§4.10).
- ☐ Docker: bin/ bind-mounted or the APK dies with the container (§6.4).
- ☐ After every build: check_apk.py RESULT: OK (§8).

11.2 What worked for me (the whole guide in one box)

Golden path — Debian 13 x86_64, 2 GB RAM + 3 GB swapfile, host Python 3.13, buildozer 1.6.0 + Cython 3.x from pip, Temurin JDK 17 tarball, run #1 to the license stop, yes-pipe license accept, the four patches, run #2 to BUILD SUCCESSFUL, check_apk RESULT: OK. Elapsed: about an hour cold, minutes warm. The Colab notebook and the Dockerfile are that exact sequence, automated.

11.3 FAQ

- Is the APK signed? Yes — debug-signed by Gradle. It sideloads on any phone. Play-Store release signing (keystore, zipalign, apksigner) is a separate, well-documented process and deliberately out of scope here.
- Windows / macOS without Docker? Use Colab (Path A) — the notebook needs only a browser.
- Can I bump the NDK / Gradle / AGP? Only if p4a's master moves first; the recipe toolchain is version-coupled. Pin to §1.2 unless you enjoy E5-class archaeology.
- Where do logs live? Wherever you tee'd them (/tmp/buildozer.log in this guide). Keep run #2's log with every release — it is the build's audit trail.
- How do I update p4a later? Delete .buildozer/android/platform/python-for-android, re-run run #1, re-accept nothing (licenses persist), re-apply patches (watch for anchor drift), run #2.

End of guide. Ship verified builds — developed by @Ariko & Arena.